

Практическая теория типов

Лекция 12: Neuro-symbolic AI & Type Theory: Trends 2023–2025

Воронов Михаил Сергеевич

ВМК МГУ

Осень 2025

«Нейросети дают интуицию, типы дают гарантии»

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические примеры и направления развития

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические примеры и направления развития

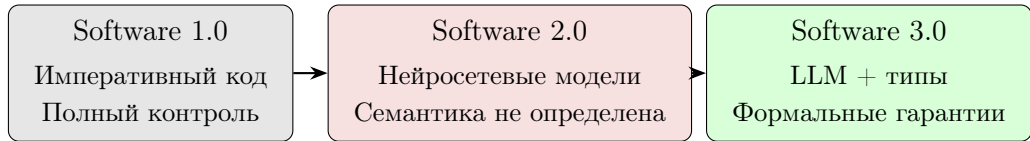
Место лекции в структуре курса

Пройденный материал:

- Лекции 1–2: λ -исчисление, редукции
- Лекция 3: Просто типизированное λ -исчисление
- Лекции 4–5: Алгебраические типы данных
- Лекция 6: System F, полиморфизм
- Лекция 7: Соответствие Карри–Ховарда
- Лекция 8: Субструктурные и сессионные типы
- Лекция 9: $\lambda\omega$, λP , зависимые типы
- Лекции 10–11: CIC, Coq/Lean, основы HoTT

Тема лекции: применение теории типов в контексте LLM и информационной безопасности.

Эволюция парадигм разработки ПО



В парадигме Software 3.0 типовые системы и пружеры верифицируют артефакты, генерируемые LLM.

Новые поверхности атаки:

- AI-ассистенты для разработки (Copilot, Cursor, Devin) в критичных репозиториях
- LLM-агенты с доступом к API и инфраструктуре
- Смарт-контракты и криптографические протоколы, генерируемые ИИ
- Финансовые системы с AI-компонентами

Теория типов и формальные методы позволяют контролировать такие системы.

С ростом возможностей AI возрастает потребность в формальных гарантиях.

Сквозной пример: WalletGuard — LLM-агент для платёжного API

Задача: агент принимает запрос на естественном языке и выполняет платёж через API банка.

Функции агента:

- парсинг → структурированная заявка
- проверка лимитов и allowlist
- 2FA / approval
- выполнение + audit trail

Угрозы:

- **format injection** (нарушение синтаксиса JSON)
- **semantic injection** (валидно, но вредно)
- нарушение порядка шагов
- double-spend / replay

Связь с лекцией 1: typestate (состояния платежа), протоколы как типы (Auth → Approve → Execute), инъекции как следствие строковых API.

LLM генерирует предложения, **типовая система** запрещает опасное.

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические примеры и направления развития

Ограничения нейросетевых подходов

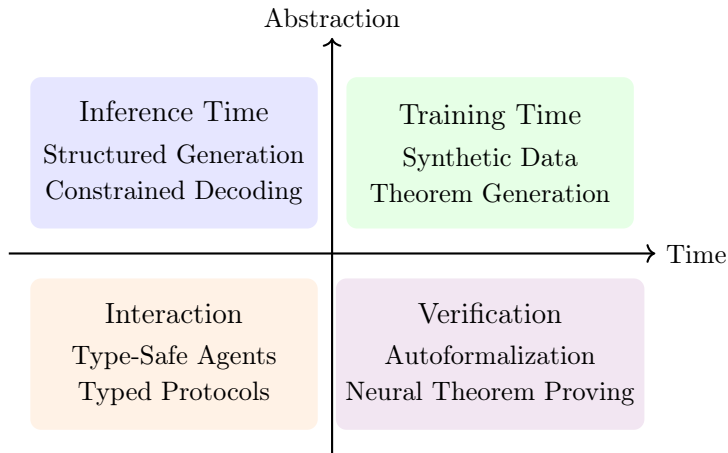
Известные проблемы LLM:

- Галлюцинации: генерация синтаксически корректного, но семантически неверного вывода
- Отсутствие формального reasoning: статистическая природа вывода
- Безопасность кода: в экспериментах Copilot генерировал уязвимые решения в ~40% CWE-сценариев (Pearce et al., 2022), пользовательские исследования подтверждают снижение безопасности (Perry et al., 2023)
- Prompt injection как новый класс атак

Neural $\neq$ Reliable

Использование LLM в критичных системах требует дополнительной валидации.

Карта исследовательских направлений



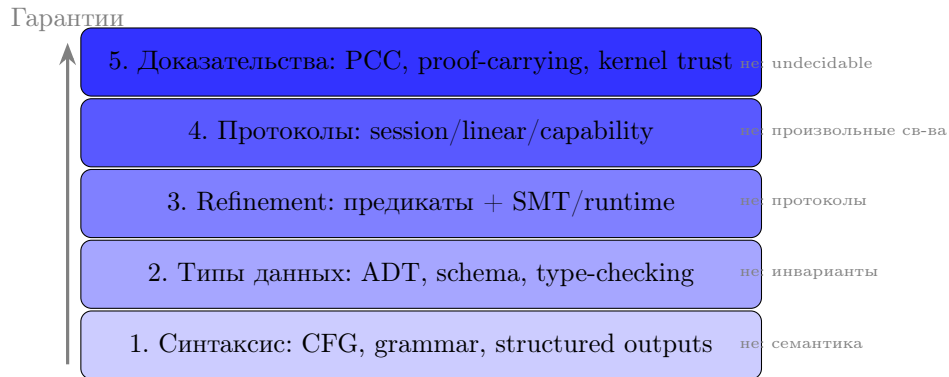
Каждое направление опирается на изученные концепции:

Концепция	Лекция	Применение в AI
ADT (суммы, произведения)	5	JSON Schema, действия агентов
Exhaustiveness checking	5	Полнота обработки действий
Curry–Howard	7	LLM ищет терм для типа
Сессионные типы	8	Протоколы агент $\leftrightarrow$ API
Линейные типы	8	Ресурсы, токены, транзакции
Σ -типы, refinement	9	Constraints, PCC
CIC, Lean/Coq	10–11	Neural theorem proving

Общий принцип: LLM генерирует термы, типовая система верифицирует.

Эта лекция — не «приложение», а кульминация курса.

Лестница гарантий



Каждый уровень добавляет гарантии, но не заменяет предыдущие.

Нейронная компонента:

- LLM (GPT, Claude, Llama)
- Reinforcement Learning
- Neural search (MCTS + NN)

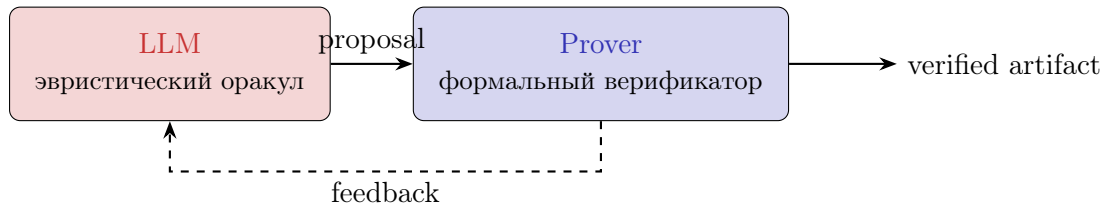
Символическая компонента:

- Формальная логика, типы
- Proof assistants
- SAT/SMT solvers

Ключевые направления 2023–2025:

- 1 Neural Theorem Proving (AlphaProof, Baldur, LeanDojo)
- 2 Autoformalization (NL $\rightarrow$ Formal Spec)
- 3 Typed Agents (Function calling, MCP)
- 4 Proof-Carrying Artifacts

Архитектурный паттерн



Паттерн: Neural Proposal $\rightarrow$ Symbolic Verification

- 1 LLM генерирует кандидата (код, доказательство, план)
- 2 Prover верифицирует, при ошибке — итеративное уточнение
- 3 При успехе — формально корректный артефакт

- ① Structured Generation — constrained decoding, грамматики, типы как спецификации
- ② Neural Theorem Proving — AlphaProof, autoformalization, proof repair
- ③ Typed Agents — function calling, MCP, сессионные типы
- ④ Code Synthesis + Proof-Carrying — feedback loops, PCC
- ⑤ Практические примеры и направления развития

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding**
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические примеры и направления развития

Проблема неструктурированного вывода

$$P(\text{token}_t \mid \text{context}) = \text{softmax}(\text{logits})$$

Языковая модель не учитывает синтаксические ограничения целевого формата.

Ожидаемый вывод:

```
{"name": "Alice", "age": 25}
```

Возможный вывод:

```
Sure! Here's the JSON:  
{"name": "Alice, "age": 25
```

Некорректный JSON приводит к ошибкам парсинга и потенциальным уязвимостям.

WalletGuard: structured output для заявки на перевод

Шаг 1 (format-level): LLM обязан выдать только объект заявки.

```
{
  "type": "object",
  "properties": {
    "to": {"type": "string"},
    "amount": {"type": "integer", "minimum": 1},
    "currency": {"enum": ["RUB", "USD"]},
    "purpose": {"type": "string"}
  },
  "required": ["to", "amount", "currency", "purpose"],
  "additionalProperties": false
}
```

LLM генерирует кандидат, **грамматика/схема** отсекает всё, что не является валидной заявкой.

Связь с ADT: схема = алгебраический тип

Вспомним лекцию 5: ADT = суммы (+) и произведения ($\times$).

JSON Schema — это в точности ADT:

"type": "object" $\cong$ произведение (record, $\times$)

"oneOf": [...] $\cong$ сумма (variant, +)

"enum": ["a" "b"] $\cong$ $1 + 1$ (конечный тип)

"required": [...] $\cong$ non-nullable (исключение None)

"items": {...} $\cong$ $\mu X. 1 + T \times X$ (список)

Принцип из лекции 5: make illegal states unrepresentable.

Structured generation реализует этот принцип на этапе генерации, а не парсинга.

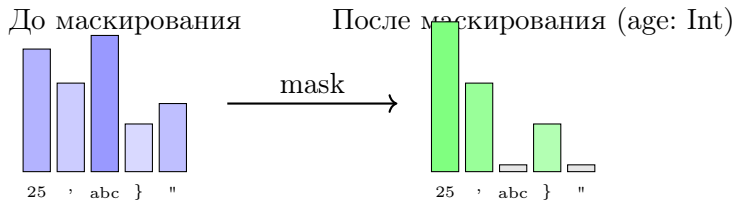
Инструменты structured generation

Outlines (dottxt-ai) — преобразование JSON Schema в конечный автомат
llama.cpp grammars — BNF-грамматики с минимальными накладными расходами
Guidance (Microsoft) — программное управление генерацией
LMQL (ETH Zürich) — декларативный язык запросов с ограничениями
SGLang (Berkeley) — язык структурированной генерации

При корректно заданной грамматике constrained decoding
гарантирует структурную валидность (JSON/AST).

Накладные расходы по latency зависят от реализации и сложности грамматики:
от заметного замедления до пренебрежимо малых в современных алгоритмах.

Механизм Logit Masking



Logit недопустимых токенов устанавливается в $-\infty$.

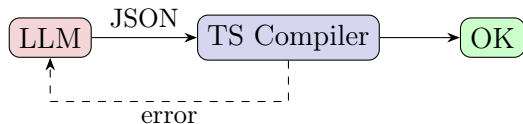
Грамматика определяет проекцию на множество допустимых продолжений.

Сравнительный анализ подходов

Критерий	Unstructured	Structured
Синтаксическая корректность	~90–95%	гарантирована*
Соответствие схеме	~70–85%	гарантировано*
Накладные расходы	базовый уровень	зависит от реализации
Обработка downstream	exception handling	гарантированный парсинг
Устойчивость к format injection	низкая	высокая
Устойчивость к semantic injection	низкая	требуется policy engine

*При корректной реализации (token boundaries, escaping).

Structured generation даёт существенное преимущество для production-систем.



Подход: использование TypeScript-интерфейсов в качестве спецификации вывода.

- 1 Определение интерфейса в TypeScript
- 2 Генерация JSON моделью
- 3 Валидация компилятором TypeScript
- 4 При ошибке — итеративное исправление

github.com/microsoft/TypeChat

Structured Outputs: промышленный стандарт (2024–2025)

Structured outputs встроены в основные LLM API:

OpenAI (2024)

- Strict JSON Schema
- Function calling

Google Gemini

- OpenAPI/JSON Schema
- Function calling

Anthropic Claude

- Tool use
- MCP

Типизированный вывод — стандарт для production LLM-систем.

WalletGuard: от Draft к VerifiedPayment через Σ -типы

Шаг 2: «черновик» превращается в «верифицированный платёж».

VerifiedPayment как Σ -тип (лекция 9):

$$\text{Verified} := \Sigma tx : \text{Draft}. \underbrace{(\text{amt}(tx) \leq \text{limit})}_{\text{лимит}} \times \underbrace{(\text{to}(tx) \in \text{whitelist})}_{\text{allowlist}} \times \underbrace{\text{KYCOK}(tx)}_{\text{compliance}}$$

Чтение: значение tx вместе с доказательствами всех ограничений.

Runtime-аппроксимация (assertions):

```
assert tx.amount <= daily_limit(user)
assert tx.to in whitelist(user)
assert kyc_check(tx) # -> VerifiedPayment
```

Structured output $\rightarrow$ форма, refinement $\rightarrow$ смысл, Σ -тип $\rightarrow$ гарантия «by construction».

Применение для защиты от инъекций

Рассмотрим поле "role": "user "admin".

Попытка атаки:

```
{"role": "user\", \"admin\": true}
```

При использовании грамматики допустимы только токены "user" или "admin".
Инъекция становится невозможной на уровне генерации.

Типизация ограничивает канал вывода.

Эшелонированная защита: типы на входе и на выходе.

Текущие ограничения реализуются как runtime assertions:

- $0 \leq \text{score} \leq 10$
- $\text{len}(\text{items}) < 100$
- $\text{date} \geq \text{today}$

Направление развития: зависимые типы для structured generation.

Длина массива, свойство отсортированности, инварианты структур данных — всё это может быть выражено в типах и верифицировано при декодировании.

См. Liquid Types (Rondon et al.), Refinement Types (Xi & Pfenning)

- Эффективные алгоритмы инкрементального парсинга
- Комбинирование синтаксических грамматик и семантических ограничений
- Типизированное декодирование для сложных структур (AST, планы агентов)
- Интеграция SMT solvers в цикл декодирования

Ключевые публикации:

- Willard & Louf (2023): Outlines
- Beurer-Kellner et al. (2023): LMQL
- Khattab et al. (2023): DSPy

Пример: Pydantic + Outlines

```
from pydantic import BaseModel
from typing import Literal
import outlines

class Person(BaseModel):
    name: str
    age: int
    role: Literal["user", "admin"]

model = outlines.models.transformers("mistralai/Mistral-7B")
generator = outlines.generate.json(model, Person)

person = generator("Create a user named Alice")
# {"name": "Alice", "age": 28, "role": "user"}
```

Результат гарантированно соответствует схеме.

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization**
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические примеры и направления развития

Проблема ручной формальной верификации

Разработка доказательств в Coq/Lean — ресурсоёмкий процесс:

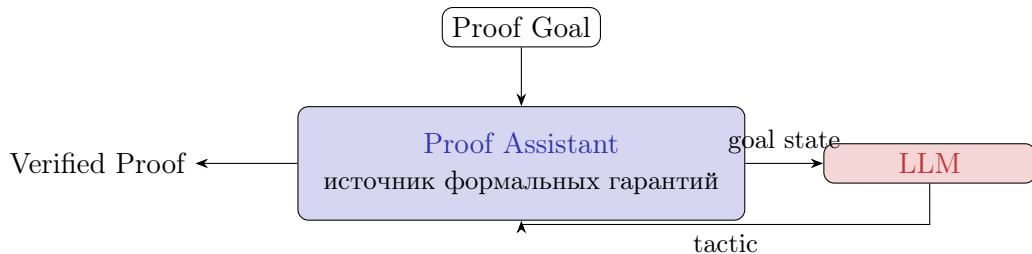
- Высокий порог входа, соотношение: 10–20 строк доказательства на 1 строку кода
- CompCert (компилятор C): ~100k строк Coq, ~6 человеко-лет¹
- seL4 (микроядро): ~200k строк доказательств в Isabelle²
- Mathlib (Lean 4): ~250k теорем, ~1.9M строк кода³

Нехватка специалистов по формальной верификации — узкое место для криптографических протоколов, смарт-контрактов, критичного ПО.

Вопрос: возможно ли применение AI для ускорения при сохранении гарантий?

¹Leroy, ERTS 2016 ²Klein et al., SOSP 2009 ³leanprover-community.github.io/mathlib_stats

LLM как генератор тактик



Цикл: Propose $\rightarrow$ Check $\rightarrow$ Repair

LLM предлагает тактику, proof assistant верифицирует, при ошибке — обратная связь.
При успехе доказательство формально верифицировано.

Curry–Howard в действии: LLM ищет терм

Вспомним лекцию 7: соответствие Карри–Ховарда.

Теорема	$\longleftrightarrow$	Тип
Доказательство	$\longleftrightarrow$	Терм (программа)
Proof Goal $\Gamma \vdash ?M : \tau$	$\longleftrightarrow$	Type Inhabitation

Что делает LLM? Генерирует терм M для заданного типа τ .

$$\text{LLM} : \Gamma \vdash ? : \tau \longrightarrow \Gamma \vdash M : \tau$$

Prover выполняет type checking: проверяет $\Gamma \vdash M : \tau$.

Это разделение генерации и верификации — ключ к надёжности.

AlphaProof и AlphaGeometry

AlphaGeometry (DeepMind, January 2024):

- Комбинация нейросети и символического решателя
- Решено 25 из 30 задач IMO-AG-30 benchmark
- Trinh et al., Nature 2024

AlphaProof + AlphaGeometry 2 (DeepMind, July 2024):

- Lean 4 + reinforcement learning + MCTS
- IMO 2024: серебряная медаль (4 из 6 задач, 28/42 баллов)
- Все решения формально верифицированы в Lean 4

Nature 2025: публикация результатов AlphaProof с полным анализом.

Нейросетевая компонента обеспечивает эвристику, proof assistant — гарантии.

Lean 4 (Microsoft Research):

- Современный proof assistant с компиляцией в C
- Mathlib: ~250k теорем, ~120k определений, ~1.9М строк кода

LeanDojo (Yang et al., NeurIPS 2023):

- Программный интерфейс LLM $\leftrightarrow$ Lean
- Benchmark: 98,734 пары теорема/доказательство
- ReProver: модель с лучшими показателями на этом бенчмарке

Связанные проекты: Pantograph, ntp-toolkit, LeanCopilot, DeepSeek-Prover

leanprover.github.io leandojo.org

Связь с CIC: почему Lean/Coq идеальны для NTP

Вспомним лекцию 10: Calculus of Inductive Constructions.

Почему CIC подходит для neural theorem proving:

- ❶ Асимметрия: проверка доказательства алгоритмична (kernel), поиск — комбинаторно сложен
- ❷ Зависимые типы: спецификации в типах ($\text{Vec } A \ n, \text{ sorted } xs$)
- ❸ Тактики: высокоуровневый язык для построения proof terms
- ❹ Kernel: маленькое доверенное ядро проверяет всё

Разделение ролей:

- **LLM** генерирует тактики (эвристика, паттерны)
- **Kernel** проверяет proof term (формальная гарантия)

Даже если LLM ошибается в 90% случаев — 10% успешных доказательств верны на 100%.

Autoformalization: NL $\rightarrow$ Formal Spec

Примеры:

NL	Formal
«Сумма чётных чётна»	$\text{even } a \wedge \text{even } b$
	$\rightarrow \text{even } (a+b)$
«Пароль зашифрован»	$\forall s. \text{enc}(s.\text{pwd})$

Workflow:

- 1 RFC/whitepaper
- 2 $\xrightarrow{\text{LLM}}$ TLA+/Lean
- 3 Верификация
- 4 Proof / counterexample

Области: OAuth, SAML, BFT/PoS, e-voting, access control policies.

Инструменты: TLA+, Tamarin, ProVerif, Lean/Coq

Autoformalization: ограничения текущих подходов

Почему autoformalization пока не является полностью автоматическим:

- Неоднозначность NL: «пользователь аутентифицирован» — какой протокол? какие гарантии?
- Скрытые предположения: неявные допущения в тексте не попадают в формализацию
- Specification drift: формальная модель расходится с реальной системой

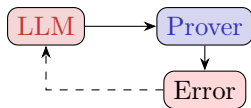
Практический workflow:

- 1 LLM генерирует черновик формализации
- 2 Эксперт выполняет валидацию и уточнение
- 3 Prover верифицирует свойства, при изменении требований — итерация

Autoformalization ускоряет процесс, но не заменяет экспертизу.

Proof Repair и Draft-Sketch-Prove

Proof Repair:



Baldur, COPRA, DeepSeek-Prover:
error message → итеративное улучшение.

Draft → Sketch → Prove:

- 1 Draft: NL-рассуждение
- 2 Sketch: скелет + sorry
- 3 Prove: формальное доказательство

Jiang et al. (2023)

Proof obligations — идеальный сигнал: бинарный, точный, формально определённый.

Синтетические данные для обучения

Проблема: объём формальных доказательств ограничен (Mathlib: $\sim 250k$ vs ImageNet: 14M).

Подход: синтетические данные + self-play

- Автоматическая генерация теорем
- Попытки построения доказательств
- Успешные примеры используются для обучения

Аналогия с AlphaZero: self-play в настольных играх $\rightarrow$ self-play в пространстве доказательств.

Polu et al. (2022), Wang et al. (2023), DeepSeek-Prover (2024) — large-scale synthetic Lean 4 data

Верификация кода:

- Verus (Rust + SMT), Creusot (Rust + Why3)
- Dafny, F*, Liquid Haskell

Автоматический синтез инвариантов: LLM генерирует кандидаты, SMT solver верифицирует.

Интеграция в CI/CD:

- 1 Код $\rightarrow$ LLM генерирует спецификацию
- 2 LLM генерирует скелет доказательства
- 3 Prover верифицирует
- 4 Слияние только при успешной верификации

Пример: Lean + LLM

```
theorem add_comm (a b : Nat) : a + b = b + a := by  
  sorry -- to be filled
```

Предложение LLM:

```
theorem add_comm (a b : Nat) : a + b = b + a := by  
  induction a with  
  | zero => simp  
  | succ n ih => simp [Nat.succ_add, ih]
```

Goal state $\rightarrow$ LLM $\rightarrow$ tactic $\rightarrow$ верификация Lean $\rightarrow$ QED

Инструменты: LeanCopilot, lean-gptf, ReProver

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы**
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические примеры и направления развития

$$\text{Agent} = \text{LLM} + \text{Memory} + \text{Tools} + \text{Goal}$$

Примеры: Cursor, Devin, Perplexity, Zapier AI

Риски:

- Некорректный вызов API → повреждение данных
- Ошибочное удаление базы данных
- Неавторизованные финансовые операции
- Prompt injection → перехват управления

Агент обладает реальными полномочиями, хотя LLM ненадёжен как источник решений.

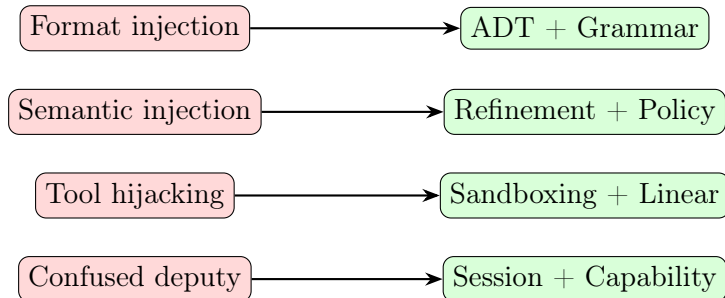
Модель угроз LLM-агента (1/2)

Класс атаки	Описание	Защита
Format injection	Нарушение структуры вывода	Grammar / structured outputs
Semantic injection	Валидное, но вредное действие	Policy engine + approvals
Tool hijacking	Подмена/отравление retrieval	Sandboxing + provenance
Confused deputy	Действие от имени жертвы	Capability-based + session types

Ключевой принцип: structured outputs защищают формат, но не семантику.

Модель угроз LLM-агента (2/2)

Где какой типовой аппарат помогает:



Эшелонированная защита (defense in depth): комбинация уровней защиты.

WalletGuard: ADT действий агента (tool calls)

Шаг 3: агент не «пишет API-вызовы», а выбирает из ADT.

```
data Action
= ParseRequest String
| GetAllowlist UserId
| GetDailyLimit UserId
| CreateTransfer Req
| RequestApproval TransferId
| ExecuteTransfer ApprovedTransfer
| LogAudit Event
```

Гарантии (лекция 5):

- **Closed world:** нет «hallucinated API» вне ADT
- **Exhaustiveness:** все действия обработаны
- **Make illegal states unrepresentable:** поля заявки обязательны по типу

Function Calling

```
{
  "name": "get_weather",
  "parameters": {
    "type": "object",
    "properties": {
      "location": {"type": "string"},
      "unit": {"enum": ["celsius", "fahrenheit"]}
    },
    "required": ["location"]
  }
}
```

Схема = тип, enum = sum type, required = non-nullable.

Поддержка: OpenAI, Anthropic, Google.

Gorilla и проблема hallucinated API

Gorilla (Patil et al., 2023): LLM, дообученная для вызова API.

Проблема: hallucinated APIs — генерация вызовов несуществующих функций.

Решение:

- ❶ Каталог API с типизированными схемами
- ❷ Retrieval релевантных API
- ❸ Constrained generation: только из каталога
- ❹ Валидация сигнатур

gorilla.cs.berkeley.edu

Model Context Protocol (MCP)

MCP (Anthropic, 2024; Linux Foundation, 2025) — открытый стандарт взаимодействия LLM и инструментов.



У каждого инструмента есть типизированная схема, вход и выход типизированы
MCP-server служит типизированным слоем между агентом и внешним миром.

Передан в Linux Foundation (2025): открытый, нейтральный, community-driven.

modelcontextprotocol.io

Переход от промптинга к программированию:

- LMQL: декларативные запросы к LLM
- DSPy: программирование с assertions
- LangChain: композиция вызовов
- LangGraph: графы состояний

Pipeline — это граф вызовов модели с типизированным состоянием.
State соответствует record type, transitions — функциям.

WalletGuard: сессионный тип протокола «создать → подтвердить → выполнить»

Шаг 4: задаём допустимый порядок шагов как session type.

```
type PayProtocol =  
  Hello .> Auth Credentials .> AuthOK  
    .> Create Req .> Quote  
    .> Approve .> ApprovedTransfer  
    .> Execute .> Receipt  
    .> Bye
```

- Нельзя выполнить Execute без ApprovedTransfer
- Нельзя пропустить Auth
- Протокол становится **типизируемым контрактом** между агентом и API

WalletGuard: линейные типы для one-shot полномочий

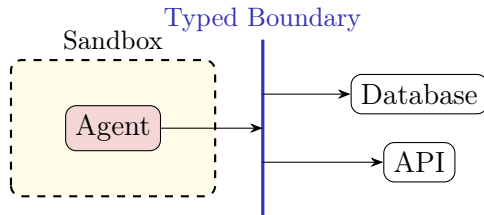
Шаг 5: подтверждение — это ресурс, который нельзя использовать дважды.

Approval token	~	линейный ресурс
Execute	~	потребление ресурса
Receipt	~	новое состояние/артефакт

$\text{execute} : \text{ApprovedTransfer} \multimap \text{Receipt}$

- **No double-spend:** токен подтверждения нельзя «скопировать»
- **No use-after-revoke:** отозванные полномочия нельзя применить
- **Protocol compliance:** хорошо сочетается с session types

Sandboxing и типизированные границы



Агент изолирован в песочнице, взаимодействие с внешним миром — через типизированные границы.

Эшелонированная защита: типизация входа, действий и выхода.

План агента — это AST, который можно статически анализировать до выполнения.

Возможные проверки:

- Типизация: корректность всех действий
- Инварианты безопасности: отсутствие запрещённых операций
- Проверка полномочий агента
- Ограничения на ресурсы: гарантия завершения

Корректный план может рассматриваться как доказательство безопасности workflow.
Применение соответствия Карри–Ховарда.

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts**
- 7 Практические примеры и направления развития

Генерация кода с помощью LLM

Масштаб: GitHub Copilot — более 20 млн пользователей (2025).

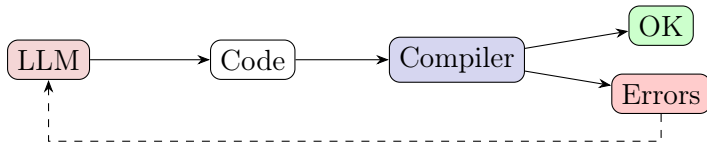
Среди пользователей Copilot ~40% зафиксированного кода — AI-generated (Microsoft, 2023).

Проблема безопасности:

- В экспериментах уязвимые решения в ~40% CWE-сценариев (Pearce et al., 2022)
- Типичные уязвимости: SQL injection, path traversal, отсутствие валидации
- Пользовательские исследования: код с AI-ассистентом менее безопасен (Perry et al., 2023)

Код для критичных систем требует дополнительной верификации.

Feedback Loops



LLM → код → компилятор → ошибки → итеративное исправление.

Self-debugging циклы улучшают точность: в экспериментах приросты до ~12%.
Эффект зависит от наличия тестов и качества сигналов обратной связи.

Chen et al. (2023): Teaching LLMs to Self-Debug

Property-Based Testing

QuickCheck/Hypothesis: генерация тестов на основе свойств.

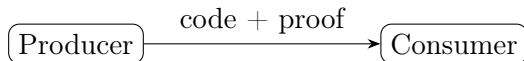
- 1 LLM генерирует код
- 2 LLM генерирует свойства (properties)
- 3 Hypothesis/QuickCheck выполняет тестирование
- 4 Контрпримеры используются как обратная связь

Properties — это частичная спецификация.

Не эквивалентно полной верификации, но превосходит unit-тестирование.

Proof-Carrying Code (PCC)

Концепция (Necula, 1997): код поставляется вместе с доказательством свойств безопасности.



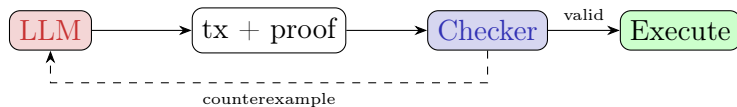
Consumer только проверяет доказательство — это быстрая операция.

Типичные свойства: memory safety, type safety, отсутствие переполнений, соблюдение политик безопасности.

WalletGuard: Proof-Carrying Transfer

Шаг 6: агент предоставляет перевод вместе с доказательством.

$$\text{PCTransfer} := \Sigma \text{tx} : \text{Transfer}. \underbrace{\text{LimitOK}(\text{tx})}_{\text{политика}} \times \underbrace{\text{WhitelistOK}(\text{tx})}_{\text{allowlist}} \times \underbrace{\text{ProtocolOK}(\text{tx})}_{\text{session type}}$$



Гарантия: проверка доказательства быстрая, генерация может быть эвристической.
Без валидного proof транзакция не исполняется.

Proof-Carrying Numbers и Data

Proof-Carrying Numbers: каждое значение сопровождается доказательством свойств.

Пример: $\text{sum} : \{n : \mathbb{N} \mid n \leq \text{limit}\}$

Proof-Carrying Data: данные с верифицируемой историей трансформаций.

Каждая операция оставляет доказательство корректности.

Применения:

- DeFi: сумма транзакции не превышает баланс
- ML pipelines: гарантии на predictions
- zk-proofs: verifiable computation

РСС через призму Σ -типов (лекция 9)

Вспомним лекцию 9: Σ -тип = зависимая пара.

$$\Sigma x : A. P(x) \cong \text{«значение } x \text{ вместе с доказательством } P(x)\text{»}$$

Proof-Carrying Code — это Σ -тип:

$$\text{PCC} = \Sigma \text{code} : \text{Program}. \text{Safe}(\text{code})$$

- Первая компонента: программа (код)
- Вторая компонента: доказательство свойства безопасности
- Невозможно создать РСС без валидного доказательства

Curry–Howard (лекция 7): доказательство $\text{Safe}(\text{code})$ — это терм.

LLM генерирует пару $(\text{code}, \text{proof})$, prover проверяет типизацию proof .

Zero-Knowledge Proofs и LLM

ZK-proofs: доказательство свойства без раскрытия данных.

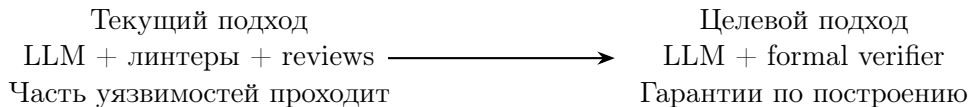
Применение LLM:

- Помощь в построении circuits
- Генерация witness
- Верификация остаётся формальной

Области применения:

- Verifiable ML inference
- Private credentials
- Blockchain bridges

От Best Effort к By Construction



Код без доказательства отклоняется. Развёртывается только верифицированный код.
Отсутствие уязвимостей определённого класса гарантируется конструктивно.

План лекции

- 1 Контекст курса и мотивация
- 2 Смена парадигмы и карта трендов
- 3 Structured Generation & Constrained Decoding
- 4 Neural Theorem Proving & Autoformalization
- 5 LLM-агенты, Tool Use и сессионные типы
- 6 Code Synthesis и Proof-Carrying Artifacts
- 7 Практические примеры и направления развития

Пример 1: AI-ассистент в банковском репозитории

Сценарий: Copilot/Cursor в кодовой базе финансовой организации.

Риски: неконтролируемые изменения, уязвимости, нарушение compliance.

Решение:

- 1 Structured generation: только типизированные патчи
- 2 Typed API boundaries: изоляция credentials
- 3 Proof-carrying patches: формальная проверка каждого PR
- 4 Audit trail: полное логирование действий AI

Пример 2: Платёжный агент (резюме WalletGuard)

Сквозной пример лекции демонстрирует все уровни защиты:

Шаг	Концепция	Лекция
1. Structured output	JSON Schema, грамматики	5 (ADT)
2. Refinement	Σ -типы, политики	9 (λP)
3. ADT действий	Closed world, exhaustiveness	5
4. Session types	Протокол Auth $\rightarrow$ Execute	8
5. Linear types	One-shot approval	8
6. PCC	tx + proof	9, 10

Итог: план агента верифицируется до выполнения, эффекты только через доказанные инварианты.

Пример 3: Верификация протокола с применением AI

Сценарий: протокол аутентификации или консенсуса.

Роль LLM:

- 1 Autoformalization: текст $\rightarrow$ TLA+ / Lean
- 2 Предложение инвариантов
- 3 Генерация скелета доказательства
- 4 Prover обеспечивает верификацию

Примеры: OAuth, TLS, Raft/PBFT, e-voting, blockchain bridges.

Инструменты: TLA+, Tamarin, ProVerif, Lean/Coq, Alloy

Рекомендуемые ресурсы

Proof Assistants:

- Lean 4 + Mathlib: leanprover.github.io
- Software Foundations: softwarefoundations.cis.upenn.edu

Structured Generation:

- Outlines: github.com/dottxt-ai/outlines
- LMQL: lmql.ai, DSPy: github.com/stanfordnlp/dspy

Neural Theorem Proving:

- LeanDojo: leandojo.org
- LeanCopilot: github.com/lean-dojo/LeanCopilot

Agents: MCP: modelcontextprotocol.io

Темы для исследовательских работ

Neuro-symbolic:

- Neuro-symbolic верификация протоколов агентов
- Autoformalization криптографического протокола (TLS/Signal)

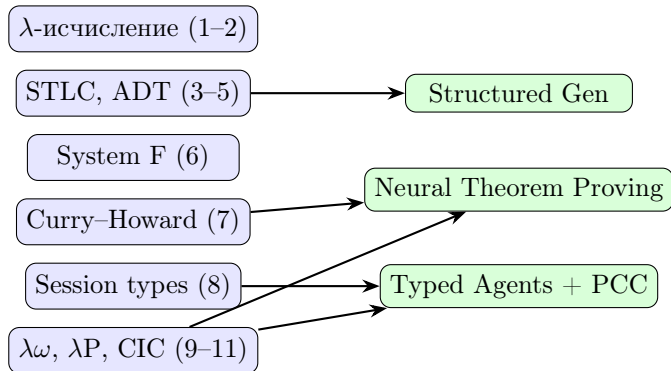
Proof-Carrying:

- PCC completions для подмножества Rust
- Верифицированный валидатор JSON Schema в Lean

Agents:

- Сессионные типы для MCP-протокола
- Type-safe agent framework с верификацией планов

Как весь курс сходится к этой лекции



Лекция 12 — не приложение, а синтез всего курса.

- Structured generation: типизация вывода LLM
- Neural theorem proving: комбинация AI и формальных доказательств
- Typed agents: безопасные LLM-агенты
- Proof-carrying artifacts: гарантии по построению

Типы обеспечивают гарантии. Нейросети обеспечивают эвристику.